The Stela Constitution: Operationalizing Stateless Governance via Dispatch Packet DP-OPS-0037

1. Introduction to Repository-Resident Autonomous Governance

The Stela platform represents a significant departure from traditional software development methodologies, moving towards a "Repository-Resident Autonomous Governance" model. In this architecture, the repository is not merely a passive storage medium for code but an active, constitutional entity that governs the behavior of both human operators and artificial intelligence (AI) agents. The central challenge addressed by Stela is "Context Saturation"—the tendency for autonomous agents to degrade in performance as the volume of information in their context window increases. To mitigate this, Stela enforces a "Stateless" doctrine where context is loaded Just-In-Time (JIT) via explicit pointers, rather than persistent memory.

This report provides an exhaustive analysis of the Stela architecture, specifically focusing on the drafting and strategic underpinnings of **DP-OPS-0037**. This Dispatch Packet (DP) acts as the enforcement mechanism for the **Stela Task Import and Porting Strategy**, a protocol designed to migrate legacy operational logic from untracked "porcelain" states into a rigorously governed, canonical framework. The implementation of DP-OPS-0037 is a critical stress test of the Policy of Truth (PoT), forcing the convergence of abstract governance rules with concrete file system operations.

1.1 The Philosophy of Statelessness and the Filing Doctrine

At the core of the Stela architecture is the **Filing Doctrine**, defined in the **Policy of Truth (PoT.md)**.[1] This doctrine organizes the repository into four distinct ontological domains, each with specific governance rules.

Domain	Directory	Function	Governance Status
Run	ops/	Executable logic, binaries, and manifests.	**Enforcement Kernel.** Must be strictly versioned and protected.

Explain	docs/	Manuals, rationale, and governance.	**Source of Truth.** Includes the Library of Agents, Tasks, and Skills.
Work	projects/	Payload code and project-specific logic.	**Payload.** Governed by PoT but distinct from the platform OS.
Trash	storage/	Local artifacts, dumps, and handoff files.	**Non-Canonical.** Never considered "Truth"; ephemeral and local-only.

The "Stateless" nature of the system dictates that an agent entering the repository has zero prior knowledge. It must "rehydrate" its understanding of the project state exclusively through the artifacts listed in the **Context Manifest** (ops/lib/manifests/CONTEXT.md).[1] If information is not present in the loaded context, it does not exist for the agent. This prevents "hallucinated" dependencies and ensures that every action is traceable to a specific file in the repository.

1.2 The Context Hazard and JIT Knowledge

A critical innovation in Stela is the identification of the **Context Hazard**.[1] The PoT explicitly forbids the inclusion of the full docs/library/ (containing Agents, Tasks, and Skills) in the global context manifest.

- **The Hazard:** If the entire library of skills were loaded into the context window for every session, the noise-to-signal ratio would become unmanageable. Agents would struggle to distinguish between relevant instructions and irrelevant background data.
- **The Solution:** Tasks and Skills are treated as **Just-In-Time (JIT)** artifacts. They are loaded only when a Dispatch Packet explicitly points to them. This "Pointer-First" methodology ensures that the AI operates with maximum focus, loading only the specific shards of knowledge required to execute the immediate objective.

1.3 The Role of Dispatch Packet DP-OPS-0037

DP-OPS-0037 is the operational manifestation of the "Stela Task Import and Porting Strategy".[1] Its objective is to inventory, classify, and port legacy task definitions—currently languishing in the untracked docs/library/tasks/imports/ directory—into the canonical **B-TASK** schema. This process is not a simple copy-paste operation; it is an "ontological transformation" that converts imperative scripts (e.g., "Run npm test") into orchestrated

delegation (e.g., "Delegate to R-AGENT-02 using tools/verify.sh").

---

2. The Constitutional Framework: Policy of Truth (PoT)

The **Policy of Truth (PoT.md)** serves as the immutable constitution of the Stela repository. It is the highest authority, overriding all other instructions, prompts, or local configurations.[1] Understanding DP-OPS-0037 requires a deep dissection of the PoT's articles, as the DP must be drafted in absolute compliance with these constraints.

2.1 Staffing Protocol and the Hierarchy of Agency

The PoT defines a strict hierarchy of agency that governs all work within the repository. This is not merely a role definition but a security constraint enforced by the ops/bin/open and repo-gates mechanisms.

- **Operator (Human):** The Operator is the sole source of "Executive Decision." They own final approvals, secrets, and the physical act of committing code to the main branch. The system is designed to be "Human-in-the-Loop" (HITL) by default; no AI agent is permitted to push directly to main.[1]
- **Integrator (Lead AI):** The Integrator acts as the "sysadmin" of the governance layer. They generate Dispatch Packets, maintain structural integrity, and audit the work of Contractors. In the context of DP-OPS-0037, the Integrator is the entity drafting the DP, analyzing the strategy, and setting the boundaries for the execution.
- **Contractor (Guest AI):** The Contractor is the execution arm. They perform the actual file edits, run the tests, and generate the proofs. They are "Guest" agents because they have no permanent standing; they are instantiated for a specific Task and terminated upon its completion.

2.2 Behavioral Logic Standard

To ensure auditability and precision, the PoT imposes a **Behavioral Logic Standard** on all AI agents.[1] This standard functions as a "cognitive style guide" that restricts the AI's natural language generation to a highly predictable, rigorous format.

- **Linguistic Precision (Contraction Prohibition):** Agents are forbidden from using contractions (e.g., "can't," "won't," "shouldn't"). They must use full forms: "cannot," "will not," "should not."
  - *Implication:* This forces a rhythmic cadence that slows down the generation process, encouraging deliberation. It also simplifies the regex patterns used by automated auditing tools (tools/lint/truth.sh) to scan for casual or imprecise language.
- **Quantitative Reporting:** Agents must report deviations in exact numerical terms whenever possible. Subjective assessments ("The code looks good") are prohibited. Acceptable output must be objective ("Execution path deviated by 0%; exit code 0").

- **Absolute Literalism:** Agents must interpret instructions with 100% fidelity. If an ambiguity exists (e.g., "Fix the bug" without specifying which bug), the agent must **STOP** and request clarification rather than inferring intent. This "Stop Condition" is a critical safety valve preventing unauthorized scope creep.

2.3 Operational Directives and Anti-Drift

The PoT establishes "Operational Directives" that serve as the immune system of the repository.

- **Anti-Drift Governance:** Any file or logic that is not aligned with the PoT coordinates is considered a "system failure." For example, if a file exists in projects/ but is not listed in docs/ops/registry/PROJECTS.md, it is "Drift." DP-OPS-0037 explicitly addresses the "Drift" caused by the untracked files in docs/library/tasks/imports/.
- **Equilibrium Maintenance:** A task is not considered complete until the system reaches "Equilibrium." This specifically means that the **State of Play (SoP.md)** history ledger must be updated to reflect the change. Code changes without a corresponding SoP entry are considered "Unlogged Mutations" and are rejected by the sop_policing workflow.[1]
- **Reuse-First Discipline:** Before creating a new artifact, the agent must cross-reference existing templates in ops/. This prevents the proliferation of redundant logic. DP-OPS-0037 enforces this by requiring the worker to check for "DUPLICATE" tasks before porting a legacy import.

2.4 Hard Constraints and SSOT

The PoT lists specific files as **Hard Constraints** or **Single Sources of Truth (SSOT)**. These files are inviolable and serve as the anchors for the entire system.[1]

- PoT.md: The Constitution.
- SoP.md: The History Ledger.
- TASK.md: The Active Work Surface.
- ops/lib/manifests/CONTEXT.md: The Context Definition.
- docs/library/MANUAL.md: The Operator Guide.
- docs/library/MAP.md: The Navigation Aid.

Any DP drafted by the Integrator, including DP-OPS-0037, must explicitly reference these constraints in its "Required Context Load" section to ensure the worker is operating within the legal framework of the repository.

---

3. The Operational Kernel: The ops/ Directory

The abstract principles of the PoT are enforced by the concrete binaries and scripts located in

the ops/ directory. These tools constitute the "Operational Kernel" of Stela. Understanding how these tools function is essential for drafting a valid Dispatch Packet, as the DP relies on them for verification and execution.

3.1 The Freshness Gate: ops/bin/open

The ops/bin/open binary is the entry point for every session. It serves as the **Freshness Gate**, ensuring that the AI is aware of the exact temporal and state coordinates of the repository.

- **Function:** It generates an "OPEN PROMPT" that includes the active branch name, the HEAD commit hash, and the status of the "Porcelain" (untracked files).
- **Forensic Verification:** When drafting DP-OPS-0037, the Integrator must extract the Branch and HEAD from the provided OPEN artifact (OPEN-work-tasks-import-port-9892660f1.txt in this case).[1]
  - **Active Branch:** work/tasks-import-port
  - **Base HEAD:** 9892660f1
- **Security Mechanism:** If the worker attempts to run the DP on a different branch (e.g., main), the open binary will report a mismatch, and the DP's internal logic (the "Freshness Gate" section) will force a **STOP**. This prevents accidental operations on the wrong codebase version.

3.2 The State Observer: ops/bin/dump

The ops/bin/dump tool allows the AI to "see" the repository state without requiring shell access or internet connectivity.

- **Determinism:** It uses git ls-files to create a deterministic snapshot of the codebase.
- **Scope Control:** The tool supports strict scoping (e.g., --scope=platform) to exclude project payloads. This is crucial for maintaining context hygiene. For DP-OPS-0037, the dump allows the worker to read the content of the legacy tasks in docs/library/tasks/imports/ and the reference tasks in docs/library/tasks/ to perform the porting operation.
- **Receipt Generation:** The dump command generates a manifest file. This manifest is a required artifact in the "Receipt" bundle. A DP is not considered complete unless the worker produces a valid dump manifest proving the final state of the system.

3.3 The Context Bundler: ops/bin/llms

ops/bin/llms acts as the "Librarian" of the repository. It compiles the canonical documentation into optimized context bundles (llms-small.txt, llms-full.txt) based on the rules defined in ops/lib/manifests/CONTEXT.md.

- **Context Hazard Enforcement:** The binary explicitly excludes hazardous directories (docs/library/agents, etc.) from the bundles. This enforcement at the binary level ensures that even if an operator forgets the rule, the toolchain prevents context saturation.
- **Freshness Metadata:** The tool injects the current HEAD hash and timestamp into the

header of llms.txt. This creates a "chain of custody" for the context, allowing auditors to verify that the AI was operating on the correct version of the documentation.

3.4 The Hygiene Engine: ops/bin/prune

Governance generates metadata. The SoP.md ledger grows with every commit. ops/bin/prune is the garbage collector of the governance layer.

- **Archive Policy:** It enforces a limit (e.g., 30 entries) on the active SoP.md file, moving older entries to storage/archives/.
- **Significance for DP-OPS-0037:** The DP must include a "Closeout" step that runs ops/bin/prune. This ensures that the extensive logging generated by the task porting process does not clog the primary ledger, maintaining system performance over time.

3.5 The Navigation Maintainer: ops/bin/map

ops/bin/map maintains the docs/MAP.md file, which serves as the "Continuity Terrain".

- **Auto-Generation:** It automatically indexes the directory structure and updates the "Auto-Generated Index" block in the map.
- **Drift Detection:** It can run in --check mode to detect if the map has drifted from the actual file system state. DP-OPS-0037 uses this to ensure that the new B-TASK files are correctly indexed in the documentation hierarchy.

---

4. The Task System Architecture: Pointer-First Constitution

The transition driven by DP-OPS-0037 is a migration from an "Imperative" legacy architecture to the "Pointer-First" constitution. This represents a fundamental shift in how the system defines "Work."

4.1 The Ontological Shift: Imperative vs. Pointer-Based

In the legacy system (represented by the files in imports/), tasks were defined imperatively. A legacy task might read:

> "Run the tests using npm test. If they fail, fix the code and try again."

This approach is fragile because it relies on **Implicit Context**. It assumes the AI knows where npm is, what constitutes a "fix," and how to interpret the output. It allows the AI too much latitude in *how* to execute the request.

In the **Pointer-First** system (represented by B-TASK-01 through B-TASK-10), the task is

defined via pointers:

> "Execute tools/verify.sh. Refer to docs/library/skills/S-LEARN-01.md for error resolution logic. Stop if exit code is non-zero."

- **Explicit Tooling:** The instruction points to a specific, version-controlled binary (tools/verify.sh) rather than a generic command. This ensures reproducibility.
- **Explicit Knowledge:** The instruction points to a specific Skill file (S-LEARN-01.md) that contains the "how-to" knowledge. This decouples the *what* (Task) from the *how* (Skill).
- **Stop Conditions:** The logic includes explicit "Stop Conditions" that force the agent to halt upon failure, preventing error cascades.

4.2 The Registry System and Ghost Artifacts

A cornerstone of the Stela governance is the **Registry System**.[1] The registries (docs/ops/registry/AGENTS.md, TASKS.md, SKILLS.md) act as the SSOT for the existence of any library artifact.

- **The Ghost Artifact Hazard:** If a file exists in docs/library/tasks/ but is not listed in docs/ops/registry/TASKS.md, it is a "Ghost Artifact." It exists on disk but is invisible to the governance layer.
- **Enforcement:** tools/lint/library.sh scans the library directories and verifies that every file is registered. DP-OPS-0037 must ensure that every ported task is immediately registered to pass this gate.

4.3 Analysis of Benchmark Tasks (B-TASK-01 to B-TASK-10)

DP-OPS-0037 instructs the worker to use the existing B-TASKs as benchmarks. An analysis of these files [1] reveals the rigid schema required:

- **B-TASK-01 (Code Review):** Demonstrates the orchestration of multiple agents (R-AGENT-02, R-AGENT-06). It separates the "Primary Agent" (who leads) from "Supporting Agents" (who assist).
- **B-TASK-02 (Verification Command):** Shows how to wrap a shell script (tools/verify.sh) in a governance layer, adding pre-flight and post-flight checks.
- **B-TASK-04 (Plan Command):** Enforces the Staffing Protocol by requiring "Explicit Approval" from the Operator before moving from planning to execution.
- **B-TASK-05 (Refactor Clean):** Mandates "Defensive State Checking" (git status --porcelain) before allowing any destructive actions (file deletion).

---

5. Strategic Analysis: The Task Import and Porting Strategy

DP-OPS-0037 is not a generic work order; it is the execution of a specific strategy document: **Stela Task Import and Porting Strategy.md**.[1] This document provides the specific algorithms the worker must use to convert "Porcelain" (untracked/legacy) into "Canon" (tracked/standardized).

5.1 The "Porcelain" Problem

The source material for this operation is the directory docs/library/tasks/imports/. The open prompt [1] identifies this directory as "Porcelain" (untracked files). This status indicates that the files are currently "Outside the Law." They may contain:

- **Contractions:** Violating the Behavioral Logic Standard.
- **Hardcoded Paths:** Violating the Filing Doctrine.
- **Duplicate Logic:** Violating the Reuse-First Discipline.

5.2 The Two-Phase Porting Protocol

The Strategy defines a rigorous two-phase process that DP-OPS-0037 must orchestrate [1]:

Phase 1: Inventory and Classification

The worker must first audit the imports. This involves:

1. **Discovery:** Listing all files in the imports directory.
2. **Read:** Analyzing the *intent* of each file.
3. **Classification:** Applying one of four tags:
    - **DUPLICATE:** The logic already exists in a B-TASK (e.g., a "Verify" script duplicates B-TASK-02). *Disposition: Delete*.
    - **OBSOLETE:** The logic references systems no longer in the stack. *Disposition: Delete*.
    - **SKILL CANDIDATE:** The file describes *how* to solve a problem (e.g., "How to debug React"). *Disposition: Harvest as S-LEARN-XX*.
    - **TASK CANDIDATE:** The file describes a workflow sequence (e.g., "Deploy to Staging"). *Disposition: Port to B-TASK-XX*.

Phase 2: The Porting Protocol (Transformation)

For files classified as **TASK CANDIDATE**, the worker performs the transformation:

1. **Synthesize Provenance:** Since legacy files lack metadata, the worker must synthesize it using the current DP-ID (DP-OPS-0037), Branch (work/tasks-import-port), and Timestamp.
2. **Define Orchestration:** The worker assigns a Primary Agent from the registry. Complex refactors get R-AGENT-05; architectural changes get R-AGENT-01.
3. **Extract Pointers:** The worker strips out imperative commands and replaces them with pointers to ops/bin/ or tools/.
4. **Define Scope Boundary:** The worker explicitly writes the "Scope Boundary" section,

limiting what the agent can touch.

5.3 Provenance Synthesis

A key challenge addressed by the strategy is the lack of history for imported files. The PoT requires provenance for all canon files. DP-OPS-0037 creates a "bridge" in the provenance chain:

- *Old Provenance:* "Unknown/Legacy"
- *New Provenance:* "Captured via DP-OPS-0037 from legacy import [Filename]."
  This ensures that the chain of custody is re-established from the moment of porting.

---

6. Constructing DP-OPS-0037: The Integrator's Logic

This section details the specific logic used to draft the Dispatch Packet. The drafting process is a simulation of the "Integrator" role, applying the constraints of TASK.md to the specific context of the request.

6.1 Defining the Freshness Gate

The first section of the DP is the Freshness Gate. This is the security lock that prevents the DP from running in the wrong context.

- **Input Data:** The user provided OPEN-work-tasks-import-port-9892660f1.txt.[1]
- **Extraction:**
  - Active branch: work/tasks-import-port
  - HEAD short hash: 9892660f1
- **Drafting Logic:** These values are hardcoded into section **3.1 Freshness Gate** of the DP. The worker is instructed to run git rev-parse commands to verify these values match the runtime environment. If they do not match (e.g., the repo has advanced since the DP was drafted), the worker must **STOP**.

6.2 Determining Required Context

The worker cannot function without context, but too much context is hazardous.

- **Standard Load:** PoT, SoP, CONTEXT, MAP. These are mandatory for every session.
- **DP-Specific Load:**
  - Stela Task Import and Porting Strategy.md: The rulebook for the specific job.
  - docs/ops/registry/TASKS.md: The destination registry (to prevent ID collisions).
  - docs/library/TASKS.md: The ledger (to log the promotion).
  - docs/library/tasks/B-TASK-01.md: A reference file to ensure the new files match the schema style.
  - docs/library/tasks/imports/: The source directory (implied access via ls command).

6.3 Configuring Scope and Safety (The Allowlist)

The **Target Files allowlist** is the most critical safety mechanism. It defines the "write permissions" for the session.

- **Inclusions:**
  - docs/library/tasks/B-TASK-*.md (New files). The wildcard allows creating multiple tasks.
  - docs/ops/registry/TASKS.md (Registry).
  - docs/library/TASKS.md (Ledger).
  - SoP.md (Governance).
  - docs/library/tasks/imports/* (Deletion).
- **Exclusions:** The DP implicitly forbids writing to ops/, tools/, or projects/ by omitting them from the allowlist. This ensures that the porting process cannot accidentally modify the platform's kernel or the application payload.

6.4 Designing the Execution Plan

The execution plan translates the Strategy's high-level goals into concrete steps.

1. **Inventory:** The first step must be discovery (ls -R).
2. **Classification & Porting:** The core logic. The DP instructs the worker to apply the "Behavioral Logic Standard" during this phase, ensuring the new files use "do not" instead of "don't."
3. **Registration:** The DP explicitly links the creation of the file to the update of the registry.
4. **Verification:** The plan mandates running tools/lint/task.sh. This script checks the syntax of the new B-TASK files. It also mandates tools/lint/library.sh to check for ghost artifacts.

6.5 The Mandatory Receipt

The DP enforces a strict "Receipt" protocol. The worker must produce:

- A **Patch/Diff** showing exactly what changed.
- A **Dump** of the platform state (ops/bin/dump).
- **Lint Results** proving that the new files adhere to the schema.
- **OPEN/PORCELAIN** artifacts proving the session started in a clean state.
  Failure to produce this receipt results in **AUTOMATIC DISAPPROVAL**.

---

7. Dispatch Packet DP-OPS-0037 (Draft Artifact)

The following is the compliant Dispatch Packet, drafted according to the strictures of TASK.md and the Stela Task Import and Porting Strategy.

DP-OPS-0037: Execute Stela Task Import and Porting Strategy

3.1 Freshness Gate (Must Pass Before Work)

Base Branch: work/tasks-import-port

Required Work Branch: work/tasks-import-port

Base HEAD: 989266Of1

Required local re-check (worker runs; paste results in RESULTS):

- git rev-parse --abbrev-ref HEAD
- git rev-parse HEAD
- git status --porcelain

Worker must already be on the Required Work Branch listed above (no branch creation or switching). STOP if any mismatch. STOP if the Required Work Branch is missing. STOP if told to create or switch branches.

3.2 Required Context Load (Read Before Doing Anything)

Worker must confirm loaded before acting:

- PoT, SoP, CONTEXT, MAP
- Plus any DP-scoped files listed here:
  - Stela Task Import and Porting Strategy.md
  - docs/ops/registry/TASKS.md
  - docs/library/TASKS.md
  - docs/library/tasks/B-TASK-01.md (Reference benchmark)

(OPEN is for Integrator/DP Writer state refresh; Worker does not read OPEN). DISPOSABLE ARTIFACTS (chat logs, etc.) MUST NOT BE REFERENCED OR

INCLUDED.

3.3 Scope and Safety

- **Objective:** Inventory legacy task imports, classify them, and port viable candidates into the canonical B-TASK pointer-first schema.
- **Non-Goals (optional, drift-killer):**
  - Do not refactor ops/ binaries.
  - Do not create new agents (use existing registry).

Target Files allowlist (hard gate)

- docs/library/tasks/B-TASK-*.md (new)
- docs/ops/registry/TASKS.md
- docs/library/TASKS.md
- docs/library/tasks/imports/* (delete/read)
- SoP.md
  Allowlist rule: exact paths only; use (new) prefix only when the DP explicitly allows new files.
- **Conditional Requirements:** If canon/governance surfaces (e.g., TASK.md) are modified, SoP.md MUST be included in the allowlist and updated.
- **Stop Condition (scope):** If any required change falls outside the allowlist, **STOP** and report.
- **Forbidden (unless DP explicitly overrides):**
  - No edits to .github/
  - No edits to projects/ unless explicitly authorized
  - No renames/moves/deletes unless explicitly authorized
- **Worker Git Authority (non-negotiable):**
  - Workers do not create/switch branches.
- Workers do not stage/commit/merge/push.
  - Workers only return diffs/results/receipt.
- **Precedence:** If any conflict exists, **PoT.md wins**.
  If unclear, STOP and ask.
- **Strict Stop Conditions (generic):**
  - STOP if any placeholder text remains in final outputs (square-bracket tokens, to-do markers, to-be-determined markers, or standalone ellipses).
- STOP if required DP inputs are missing (do not guess).
- STOP if IDs / naming constraints in the DP are inconsistent (when applicable).

3.4 Execution Plan (A-E Canon)

3.4.1 State

- **Context:** Legacy task definitions currently exist in docs/library/tasks/imports/ as untracked "porcelain" files. These files likely contain imperative logic and hardcoded paths that violate the Pointer-First Constitution.
- **Drift:** The presence of untracked logic creates a "Ghost Artifact" hazard where operational knowledge exists outside the Registry (SSOT).
- **Desired State:** All viable legacy tasks are ported to docs/library/tasks/ following the strict B-TASK schema, registered in docs/ops/registry/TASKS.md, and the legacy imports directory is cleared.

3.4.2 Request

Numbered tasks (concrete, no vibes):

1. Run ls -R docs/library/tasks/imports/ to inventory candidate files.
2. For each file, read the content and classify as **DUPLICATE** (delete), **OBSOLETE** (delete), or **TASK CANDIDATE** (port).
3. For **TASK CANDIDATE** files, synthesize provenance (using current DP-ID and timestamp) and map the legacy logic to the "Orchestration" and "Pointers" sections of the B-TASK schema.
4. Create the new B-TASK files in docs/library/tasks/ (e.g., B-TASK-11.md, B-TASK-12.md), ensuring strict adherence to the "Behavioral Logic Standard" (no contractions, quantitative reporting).
5. Register the new tasks in docs/ops/registry/TASKS.md and append entries to the Task Promotion Ledger in docs/library/TASKS.md.
6. Delete the ported/obsolete files from docs/library/tasks/imports/.
7. If blocked by ambiguous legacy logic, STOP and use BLOCKED shape.

Required Content (optional; use when doc/governance changes demand exact text/format):

- New B-TASK files must include ## Provenance, ## Orchestration, ## Pointers, ## Execution Logic, and ## Scope Boundary sections.
- Execution Logic must use pointers (e.g., tools/verify.sh) instead of raw commands (npm test).

3.4.3 Changelog

- **Log (human-readable, 1–6 bullets):**
  - Ported legacy imports to canonical B-TASK schema.
  - Updated Task Registry and Promotion Ledger.
  - Removed legacy import artifacts.

3.4.4 Patch / Diff

- **Format:** Unified diff (preferred) or anchored snippets.

3.4.5 Receipt (Required)

Verification (MUST RUN; or report NOT RUN + reason + risk)

- ./ops/bin/dump --scope=platform (or repo-correct equivalent when scope differs)
- If dump refiners are used, include this fallback line in the DP:
  - Fallback:./ops/bin/dump --scope=platform --format=chatgpt --out=auto --bundle
- **Zero-Byte Check:** Verify dump is not empty (test -s <path>).
- Context lint (use repo-canonical command):
  - Prefer: bash tools/lint/context.sh
  - If not found and required: **STOP** and ask (do not invent a substitute)
- Truth lint (when PoT or canon surfaces touched; use repo-canonical command):
  - Prefer: bash tools/lint/truth.sh
  - If not found and required: **STOP** and ask (do not invent a substitute)
- **Task Lint:** bash tools/lint/task.sh (Required for new B-TASKs).
- **Library Guard:** bash tools/lint/library.sh.

Skill capture (required during normal DP processing)

- Workflow: run ops/lib/scripts/skill.sh harvest to auto-generate a draft with provenance and semantic collision checks.
- Review: inspect the draft in storage/handoff/ for heuristic accuracy (Hot Zone and High Churn). Refine Invocation guidance and Solution.
  Do not edit the Provenance block.
- Promote: run ops/lib/scripts/skill.sh promote <draft_path> or log "no new skill promoted" with a rationale in RESULTS.
- Legacy capture is forbidden. Do not manually write skill files; the provenance block is required.
- If a DP requires skill capture, the DP Target Files allowlist must include docs/library/SKILLS.md.
- Proof required in RESULTS:
  - Harvest-only: command invoked plus draft path and ls storage/handoff/ output.
- Promotion: command invoked plus grep -n "Promotion Packet:" docs/library/SKILLS.md and grep -n "docs/library/skills/S-LEARN-" docs/library/INDEX.md.

Artifact bundle (required directory)

- storage/handoff/

Proof bundle checklist (required; naming pattern over exact names)

- storage/handoff/DP-OPS-0037-RESULTS.md
- storage/handoff/OPEN-PORCELAIN-work-tasks-import-port-9892660f1.txt
- storage/handoff/OPEN-work-tasks-import-port-9892660f1.txt

- storage/dumps/dump-full-YYYY-MM-DD-HHMMSS.txt
- git diff --name-only (include in RESULTS)
- git diff --stat (include in RESULTS)
- NEXT: one single action (include in RESULTS)

MANDATE: No receipt/proof bundle = **AUTOMATIC DISAPPROVAL**.

Closeout routine receipts not being produced CONSISTENTLY INCLUDING PORCELAIN, DUMP AND OPEN is a failure condition.

Partial receipts (e.g., missing OPEN artifacts when a session was started) are considered failures.

REQUIRED OUTPUT

MUST include a "Status" block at the top of RESULTS:

- Scope summary: (1 sentence validating the loop)
- Tracked change: (What was edited)
- Verification: (Output of proofs)

RESULTS file must include (minimum)

- Verification results (RUN/NOT RUN + reason + risk)
- Patch/diff proof bundle (status/diff list or direct diff)
- Receipt pointers (OPEN + PORCELAIN + DUMP outputs/paths)
- git diff --name-only output
- git diff --stat output
- NEXT: one single action

Storage artifact flow (local-only, untracked):

- OPEN and OPEN-PORCELAIN outputs land in storage/handoff/.
- Dump bundles land in storage/dumps/dump-full-YYYY-MM-DD-HHMMSS.txt (example; use actual dump filename).
- Results receipts land in storage/handoff/DP-OPS-0037-RESULTS.md.
- Operator stores DP drafts under storage/dp/intake/ and moves or copies to storage/dp/processed/ after completion (or when reusing a previously executed DP).
- These are retention paths only; they are not worker prerequisites unless the DP explicitly says so.

---

4. Closeout (Mandatory)

Worker closeout duties (end of session)

- Produce or update: storage/handoff/DP-OPS-0037-RESULTS.md.
- If canon/governance surfaces were changed, include **SoP.md** in scope and update it in the same PR slice.
- Run ops/bin/prune (hygiene).
- Run ops/bin/llms (refresh context bundles).
- Ensure proof bundle checklist is satisfied (above).
  (No bundle = DISAPPROVE.)
- Fill out the Mandatory Closing Block (see below).
- Append a **TASK Work Log** entry (below) capturing:
  - what changed (high level)
  - gates/verification outcome (pass/fail or not run + reason)
  - blockers (if any)
  - **NEXT:** one single action

Mandatory Closing Block (Varied Wording Required)

Varied Wording provision: Each entry must use meaningfully distinct wording;

copy or minor tense changes are not acceptable. Entry 4 must differ from Entry 1.

1. Primary Commit Header (plaintext)
2. Pull Request Title (plaintext)
3. Pull Request Description (markdown)
4. Final Squash Stub (plaintext) (Must differ from #1)
5. Extended Technical Manifest (plaintext)
6. Review Conversation Starter (markdown)

Integrator/operator closeout duties

- Review diff vs DP scope/forbidden zones.
- Run repo-gates / required checks.
- Commit/push/open PR/merge (operator-controlled).

4.1 Thread Transition (Reset / Archive Rule)

Purpose: Prevent TASK from becoming a museum while preserving continuity.

When a DP is **complete** (merged) or **ended** (canceled/superseded):

1. Add a final Work Log entry that starts with: THREAD END: and includes:
   - outcome (merged/canceled/superseded)
   - pointer to the DP RESULTS file
   - NEXT: the single action (often “start new DP” or “await operator”)
2. Reset sections **2. Active Context** and **3. Current Dispatch Packet** to the next DP.
3. Keep the Work Log, but separate threads with a visible divider line:

- ---
- THREAD START: DP-OPS-0000 (use the actual next DP id)

(If the Work Log gets too long, it's acceptable to keep only the most recent thread's entries here and move older thread logs into the DP RESULTS file—only when explicitly directed.)

---

5. Work Log (Timestamped Continuity)

> **NOTICE:** This log is for the *active* DP only.
>
> Historical logs are archived in 'storage/archives/'. Do not rely on this log for long-term history.
>
> **Rule:** This is not a transcript. It is the durable breadcrumb trail.
>
> **Expectation:** Append a new timestamped entry on every DP closeout (no exceptions).
>
> **Each entry ends with:** NEXT: <one single action>.

---

8. Technical Deep Dive: Enforcement Mechanisms and Linter Architecture

The robustness of DP-OPS-0037 depends on the "Immune System" of the repository: the Linter Suite (tools/lint/). This section analyzes how these tools technically enforce the high-level governance described in the PoT.

8.1 tools/lint/task.sh: The Schema Validator

This script is the primary enforcement mechanism for the Pointer-First Constitution within the task library.[1] It operates by parsing every markdown file in docs/library/tasks/.

- **Section Enforcement:** It scans for the required headers: ## Provenance, ## Orchestration, ## Pointers, ## Execution Logic, ## Scope Boundary. If any are missing, the build fails. This ensures that no "lazy" tasks are committed.
- **Semantic Ambiguity Detection:** The script uses regex to find ambiguous imperative verbs like "check," "verify," "ensure," or "validate" in the Execution Logic section. If these words appear *without* being enclosed in backticks (indicating a file pointer) or associated with a tool, the linter flags a failure: *"Task contains narrative execution language without pointers."* This forces the worker to write "Run tools/verify.sh" instead of "Verify the code."

8.2 tools/lint/library.sh: The Registry Guard

This script [1] enforces the **Registry System**. It performs a bidirectional check:

1. **Dead End Check:** It verifies that every ID listed in docs/ops/registry/TASKS.md corresponds to an actual file in docs/library/tasks/.
2. **Ghost Check:** It verifies that every file in docs/library/tasks/ has a corresponding entry in the registry.
   This mechanism prevents the "Shadow IT" problem, where operational logic exists in the repo but is not tracked by the governance layer. For DP-OPS-0037, this means the worker *must* update the registry in the same commit as creating the new B-TASK files, or the PR will be rejected.

8.3 tools/lint/truth.sh: The Canon Defender

This script [1] protects the core terminology and spelling of the platform.

- **Forbidden Spellings:** It scans for typos of "Stela" (e.g., "Stella," "Stila") and legacy terms like "Sheriff" or "Colonies" that might persist from previous iterations of the governance model.
- **Scope:** It scans all authored surfaces (ops, docs, tools, .github). This ensures that the "brand integrity" of the governance model is maintained across the entire stack.

8.4 tools/lint/context.sh: The Context Validator

This script [1] ensures that the **Context Manifest** (ops/lib/manifests/CONTEXT.md) is valid.

- **Manifest Supremacy:** It verifies that every file listed in the manifest actually exists.
- **Hazard Detection:** Crucially, it scans the manifest for "Context Hazards"—references to docs/library/agents, tasks, or skills. If it detects any of these directories in the global context load, it triggers a critical failure. This technically enforces the JIT philosophy.

---

9. Execution Simulation: The Life of DP-OPS-0037

To fully understand the impact of this DP, we must simulate its execution lifecycle from the perspective of the "Contractor" (Guest AI).

9.1 Phase 1: Ingestion and Verification

1. **Startup:** The Contractor initiates the session using the ops/bin/open prompt. It reads the PoT, SoP, and TASK files.
2. **Gate Check:** It reads DP-OPS-0037. It runs git rev-parse HEAD. It confirms the hash matches 9892660f1. It confirms the branch is work/tasks-import-port. The Freshness Gate passes.
3. **Context Loading:** It explicitly acknowledges loading the extra files: the Strategy, the

Registry, and the Ledger.

9.2 Phase 2: The Porting Loop

1. **Inventory:** It runs ls -R docs/library/tasks/imports/. It finds a file, say deploy-script.md.
2. **Classification:** It reads deploy-script.md. It sees imperative commands: "Run npm run build then scp to server." It tags this as a **TASK CANDIDATE**.
3. **Drafting:** It creates docs/library/tasks/B-TASK-11.md.
   - *Provenance:* Captured 2026-02-08 via DP-OPS-0037.
   - *Orchestration:* Primary Agent: R-AGENT-04.
   - *Pointers:* It replaces npm run build with tools/build.sh (assuming it exists) or notes a gap.
   - *Logic:* "1. Execute tools/build.sh. 2. Execute tools/deploy.sh."
4. **Registration:** It appends | B-TASK-11 | Deploy Script | docs/library/tasks/B-TASK-11.md | | to docs/ops/registry/TASKS.md.
5. **Logging:** It appends the event to docs/library/TASKS.md.

9.3 Phase 3: The Verification Gauntlet

1. **Self-Correction:** The Contractor runs tools/lint/task.sh. It might fail if the Contractor used the word "Check" in step 3. The Contractor rewrites it to "Execute tools/check.sh."
2. **Ghost Check:** It runs tools/lint/library.sh. The check passes because the registry was updated.
3. **Dump:** It runs ops/bin/dump to generate the final state proof.

9.4 Phase 4: Closeout

1. **SoP Update:** The Contractor realizes it modified Canon (the Task Library). It appends an entry to SoP.md: *"2026-02-08 — DP-OPS-0037 Task Porting. Ported legacy deploy script to B-TASK-11 and updated registry."*
2. **Prune:** It runs ops/bin/prune, archiving old SoP entries.
3. **Handoff:** It packages the results into storage/handoff/DP-OPS-0037-RESULTS.md.

---

10. Conclusion and Strategic Outlook

The drafting and execution of **DP-OPS-0037** serves as a definitive proof-of-concept for the Stela Pointer-First Constitution. It demonstrates that complex, legacy, imperative logic can be successfully migrated into a stateless, governed architecture without loss of fidelity.

10.1 The Triumph of Structure over Prompting

The Stela architecture proves that "Prompt Engineering" is an insufficient mechanism for robust AI governance. Instead, **Context Architecture**—the combination of Allow lists, Freshness Gates, and Linters—provides a deterministic framework where AI behavior is

constrained by "Physical Laws" (the PoT) rather than suggestion.

10.2 The End of Drift

By treating "Drift" (the divergence of documentation from reality) as a system failure, Stela solves the problem of "Rot" that plagues most software documentation. The system forces synchronization. A task cannot exist unless it is registered; it cannot be registered unless it exists. This tautology, enforced by tools/lint/library.sh, guarantees the long-term integrity of the knowledge base.

10.3 Future Implications

The successful execution of the Task Import Strategy paves the way for the next phase of Stela's evolution: **Recursive Optimization**. Once all tasks are canonical B-TASKs, the Integrator can begin to analyze the library for redundancy, merging similar tasks and optimizing agent orchestration paths. The "Porcelain" will be clean, and the "Canon" will be Truth.

This report confirms that the draft of DP-OPS-0037 is not only compliant with all constraints but is the optimal instrument for achieving the repository's strategic goals.

Works cited

1. dump-full-work-tasks-import-port-9892660f1.txt